Name: Cohort/Batch: Date: Score:

GOOGLE GKE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A DevOps

TOTAL: 10 QUESTIONS

Q1 What problem does Google GKE solve in DevOps or infrastructure? — Threat Model

Q2 What is Google GKE's core architecture? — Architecture

Q3 How does Google GKE define infrastructure or configuration as code? — Configuration

Q4 How does Google GKE ensure idempotency? — Hardening

Q5 How do you handle secrets in Google GKE? — SSO / IdP

Q6 How does Google GKE integrate with CI/CD pipelines? — Observability

Q7 How do you debug a failed Google GKE deployment? — Lifecycle

Q8 What are security best practices for Google GKE? — Compliance

Q9 How does Google GKE scale for large environments? — Testing

Q10 What are common mistakes teams make when adopting Google GKE? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Google GKE addresses a specific concern

Mitigation

Google GKE addresses a specific concern provisioning, configuration management, orchestration, or CI/CD by automating manual steps that are error-prone, slow, or not reproducible when done by hand.

A2 Google GKE has a control plane

Primitives

Google GKE has a control plane that stores desired state and a data plane (agents or push-based SSH) that enforces it. Understanding this distinction clarifies where configuration is evaluated and where changes are applied.

A3 Google GKE uses declarative files (YAML,

Hardening

Google GKE uses declarative files (YAML, HCL, or a DSL) to express desired state. A plan/diff phase shows what will change before applying. Version-controlling these files enables peer review and rollback.

A4 Google GKE operations are designed to

Gotchas

Google GKE operations are designed to produce the same result when run multiple times. Applying the same config twice converges to the desired state rather than creating duplicate resources. This makes automation safe to re-run.

A5 Secrets should never be stored in

Federation

Secrets should never be stored in plain text in Google GKE config files. Use a secrets backend (Vault, AWS Secrets Manager) and inject values at runtime via environment variables or encrypted vaults supported by the tool.

A6 CI/CD pipelines call Google GKE in

Observability

CI/CD pipelines call Google GKE in automated steps: plan on a pull request to preview changes and apply on merge to main. Approval gates can require a human to review the plan before changes go to production.

A7 Check Google GKE's logs for the

Automation

Check Google GKE's logs for the specific error message and the resource that failed. For infrastructure tools, re-run with increased verbosity (-v, --debug). Review the state file or event history to understand what changed before the failure.

A8 Rotate credentials used by Google GKE,

Governance

Rotate credentials used by Google GKE, restrict network access to its control plane, enable audit logging of all operations, and use least-privilege service accounts. Never run Google GKE with admin credentials in production.

A9 Large Google GKE deployments use

Assessment

Large Google GKE deployments use workspaces or namespaces to isolate environments, remote state backends for team collaboration, and modular code to avoid a single monolithic config that is slow to plan/apply.

A10 Common pitfalls include storing secrets in

Optimization

Common pitfalls include storing secrets in plain text config, skipping the plan step before apply, not reviewing state drift, and not having a rollback strategy when a deployment fails partway through.